

The University of Lahore
Department of Computer Science & IT
Final Term Examination



Spring-2024

Course Title:	Introduction to Software Engineering	Course Code:	CS-04303	Credit Hours:	3
Course Instructors:	Ms. Saman Abdullah, Ms. Sadia Hameed, Ms. Komal Ali, Mr. Usman Rasheed	Program Name:	BSCS		
Time Allowed:	120 Minutes	Maximum Marks:	50		
Date:	15-05-2024	Course Mentor:	Mr. Atif Ikram		
Student's Name:		Signature:			

Objective Part (overwriting and cutting is not allowed)

Time: 15 Minutes

Question # 1: Circle the correct option.

(10 Marks)

- What is the primary purpose of system modeling?
 - Developing software code
 - Represent visual design elements
 - Understand and analyze system requirements
 - Test system components
- In system modeling, what does the "context model" primarily focus on?
 - Detailed process flow
 - System boundaries and external entities
 - Class diagrams
 - Use case scenarios
- In use case modeling, what does a "use case" represents?
 - System boundaries
 - External entities
 - A specific functionality or interaction
 - Class diagrams
- In the context of UML, what does "aggregation" indicate between two classes?
 - Strong association
 - Temporary relationship
 - Whole-part relationship
 - Inheritance relationship

5. In event-driven modeling, what triggers the execution of specific actions?
 - A. User input
 - B. Control flow
 - C. Class relationships
 - D. System boundaries
6. In model-driven engineering, what are the high-level models typically expressed in?
 - A. Assembly language
 - B. UML(Unified Modeling Language)
 - C. Machine code
 - D. HTML(Hypertext Markup Language)
7. Which architectural pattern divides an application into layers such as presentation, business logic, and data access?
 - A. MVC pattern
 - B. Layered architecture
 - C. Repository architecture
 - D. Client-Server architecture
8. In the context of software architecture, what does "MVC pattern" stand for?
 - A. Model View Container
 - B. Model View Controller
 - C. Multi-View Configuration
 - D. Modular View Component
9. What is the primary goal of "validation" in software testing?
 - A. Identifying defects
 - B. Ensuring that the software meets specified requirements
 - C. Executing test cases
 - D. Debugging software
10. In the software testing process, what is the purpose of "system testing"?
 - A. Testing individual components
 - B. Verifying the behavior of the entire system
 - C. Identifying syntax errors
 - D. Executing test cases for a specific module

Spring-2024

Course Title:	Introduction to Software Engineering	Course Code:	CS-04303	Credit Hours:	3
Course Instructors:	Ms. Saman Abdullah, Ms. Sadia Hameed, Ms. Komal Ali, Mr. Usman Rasheed	Program Name:	BSCS		
Time Allowed:	120 Minutes	Maximum Marks:	50		
Date:	15-05-2024	Course Mentor:	Mr. Atif Ikram		
Student's Name:		Signature:			

Subjective Part

Time: 105 Minutes

Total Marks: 40

Scenario:

A vending machine sells small, packaged, ready to eat items (chocolate bars, cookies, candies, etc.). Each item has a price and a name. A customer can buy an item, using a smart card (issued by the vending machine company) to pay for it. No other payment forms (i.e. cash, credit card) are allowed. The smart card records on it the amount of money available.

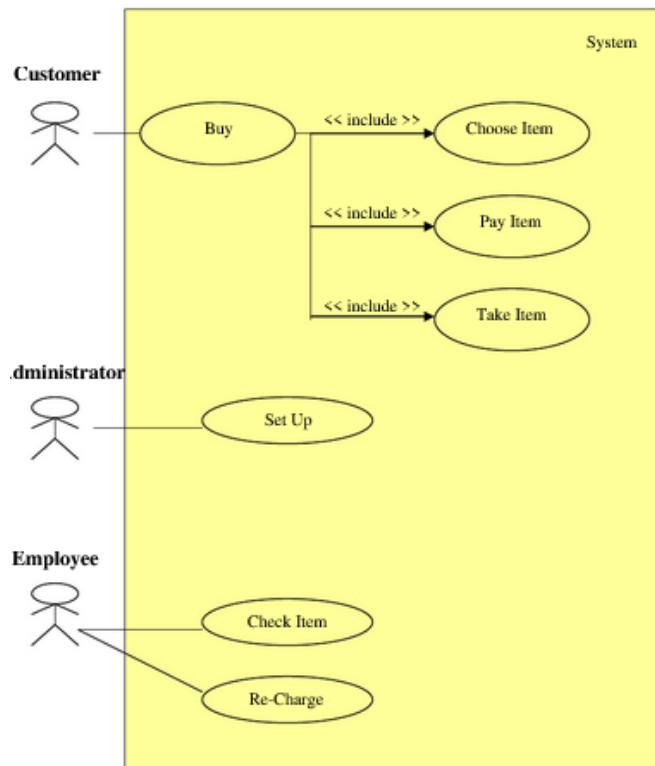
The functions supported by the system are:

- Sell an item (choose from a list of items, pay item, distribute item)
- Recharge the machine
- Set up the machine (define items sold and price of items)
- Monitor the machine (number of items sold, number of items sold per type, total revenue)

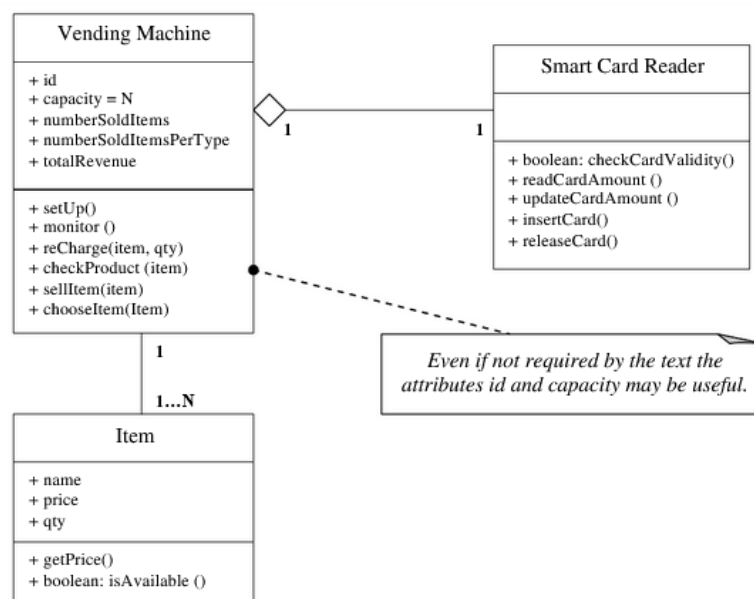
The system can be used by a customer, a maintenance employee (who recharges items in the machines), and an administrator (who sets up the machine).

Question # 2:**(8 Marks)**

Draw the use case diagram for the above-mentioned scenario. You'll have to make a number of assumptions about the manner in which a user interacts with this system.

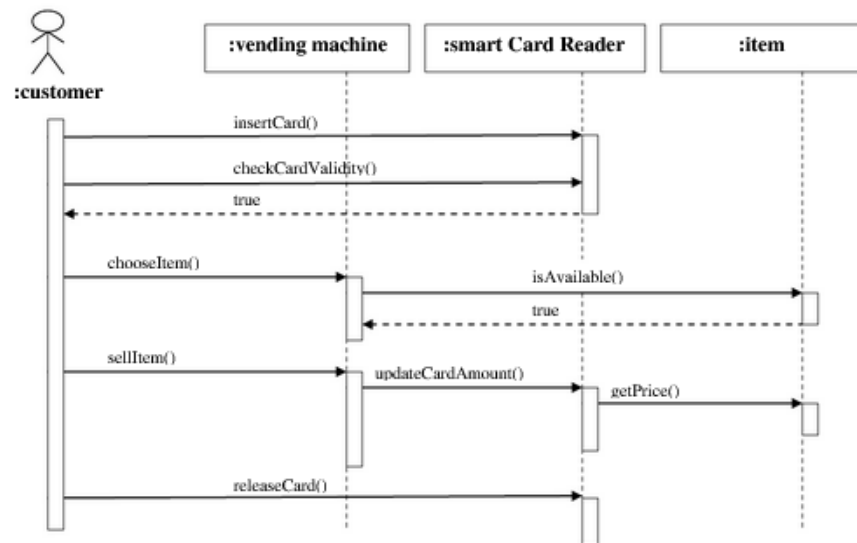
**Question # 3:****(8 Marks)**

Identify the classes, attributes and their relationships from the above-mentioned scenario and draw a class diagram.



Question # 4:**(8 Marks)**

Enrich the model of the Vending Machine with a sequence diagram describing a successful sales procedure.

**Question # 5:****(8 Marks)**

After implementing new features and updates in a mobile application, the development team wants to ensure that previously working functionalities remain intact. They decide to conduct regression testing to verify that the recent changes haven't introduced any unintended defects or caused existing features to malfunction.

Which type of testing is most suitable for identifying and fixing such defects?

SOLUTION:

Regression testing is a type of software testing that is performed to ensure that recent changes or additions to a software application haven't adversely affected its existing functionality. It involves re-executing previously conducted test cases to verify that the previously working features still perform as expected after the introduction of new code or modifications.

Here's an overview of how regression testing works:

Identify Test Cases: The first step in regression testing is to identify the test cases that cover the critical functionalities of the application. These test cases represent the various scenarios and user interactions that need to be validated.

Execute Test Cases: Once the test cases are identified, they are executed against the updated version of the software. This involves running the same tests that were previously conducted to verify the functionality of the application.

Compare Results: The test results from the current regression testing are compared with the results obtained from the previous testing cycles. Any discrepancies or failures indicate potential defects introduced by the recent changes.

Investigate Failures: If any test cases fail during regression testing, the development team investigates the cause of the failure. They identify whether the issue is due to the recent changes or if it's an existing defect that has resurfaced.

Fix Defects: Once the cause of the failure is identified, developers fix the defects introduced by the recent changes. This may involve modifying the code, updating configurations, or addressing any other issues that caused the regression.

Re-Run Tests: After fixing the defects, the regression tests are re-run to ensure that the changes have been successfully implemented and that the previously failing test cases now pass.

Repeat: Regression testing is an iterative process that may need to be repeated multiple times throughout the development lifecycle, especially after each new feature or update is introduced.

The primary goal of regression testing is to maintain the stability and reliability of the software application by ensuring that new changes do not unintentionally break existing functionality. It helps in identifying and addressing defects early in the development process, thus minimizing the risk of introducing issues into the production environment.

Regression testing is the most suitable type of testing for identifying and fixing defects that may have been introduced by recent changes or updates in a mobile application. This type of testing involves re-running previously executed test cases to ensure that existing functionalities still work as expected after new changes have been implemented. By comparing the current behavior of the application with its previous behavior, regression testing helps in detecting any unintended defects or regressions caused by the recent updates.

Question #6:

(8 Marks)

Explain the concept of architectural patterns and their significance in architectural design. Provide examples of commonly used architectural patterns, such as MVC (Model-View-Controller) and Layered Architecture, and discuss their advantages and limitations?

SOLUTION:

Architectural patterns are high-level design patterns that provide commonly used solutions to architectural problems in software development. They offer a blueprint for organizing the structure, behavior, and interactions of software systems, guiding developers in designing robust, scalable, and maintainable applications. These patterns abstract away specific implementation details and focus on defining the overall structure and relationships between components within a system.

Model-View-Controller (MVC):

Description: MVC separates an application into three interconnected components: Model, View, and Controller. The Model represents the data and business logic, the View displays the user interface, and the Controller handles user input and updates the Model and View accordingly.

Significance of Architectural Patterns:

Standardization: Architectural patterns provide standardized solutions to common architectural problems, fostering consistency and coherence in software design across different projects and teams.

Scalability: They enable the development of scalable systems by defining modular and loosely coupled architectures that can accommodate growth and changes in requirements.

Maintainability: Architectural patterns promote separation of concerns and modularity, making it easier to understand, extend, and maintain software systems over time.

Reusability: By encapsulating best practices and design principles, architectural patterns facilitate reusability of design solutions across multiple projects.

Flexibility: They allow developers to adapt and evolve the architecture of a system to accommodate new requirements or technological advancements without requiring extensive redesign.

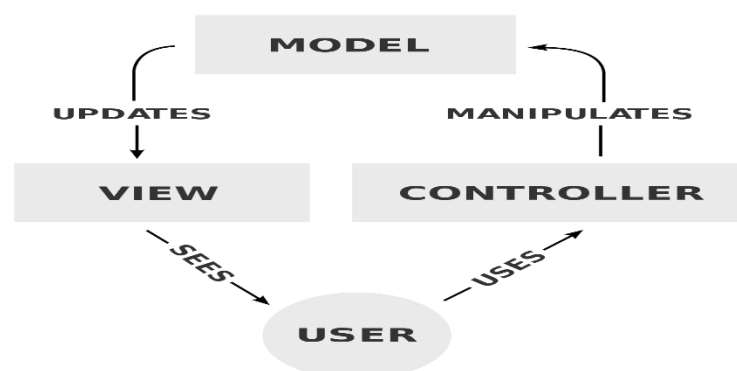
MVC separates application into three components:

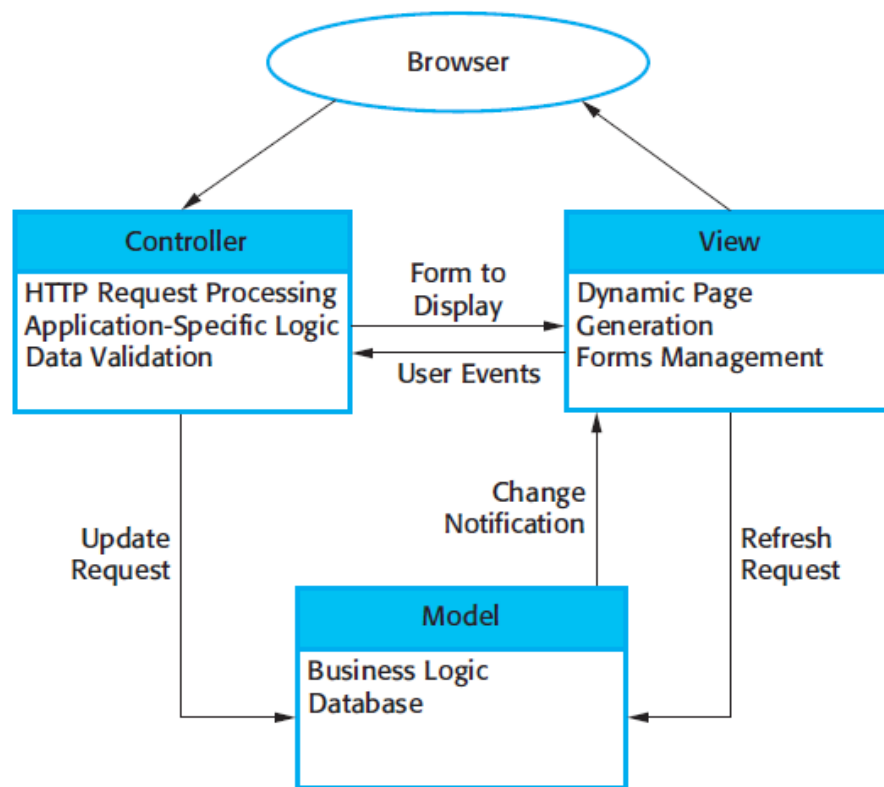
Model: represents shape of the data and business logic.

View: is a user interface. View displays data using model to the user and also enables them to modify the data.

Controller: Controllers are the components that handle user interaction, work with the model, and ultimately select a view to render that displays UI.

In an MVC application, the view only displays information; the controller handles and responds to user input and interaction.





Description: MVC separates an application into three interconnected components: Model, View, and Controller. The Model represents the data and business logic, the View displays the user interface, and the Controller handles user input and updates the Model and View accordingly.

Advantages: Promotes separation of concerns, enhances maintainability, allows for multiple views of the same data, and facilitates parallel development by decoupling components.

Limitations: Complexity can increase with larger applications, may lead to overuse of callbacks or event-driven programming, difficult to maintain consistency between views.

Layered architecture:

- Used to model the interfacing of sub-systems.
- Organises the system into **a set of layers**, each of which provide a set of services.
- Each layer only relies on the facilities and services offered by the layer immediately beneath (below) it.
- Supports the incremental development of sub-systems in different layers. When a layer interface changes, only the adjacent layer is affected.

User Interface

User Interface Management
Authentication and Authorization

Core Business Logic/Application Functionality
System Utilities

System Support (OS, Database etc.)

Example:

Web Browser Interface

LIBSYS
Login

Forms and
Query Manager

Print
Manager

Distributed
Search

Document
Retrieval

Rights
Manager

Accounting

Library Index

DB1

DB2

DB3

DB4

DBn

Layered Architecture:

Description: Layered architecture divides an application into distinct layers, each responsible for a specific aspect of functionality. Common layers include Presentation/UI layer, Business layer, and Data Access layer.

Advantages: Encourages modularity and separation of concerns, promotes reusability of components, facilitates parallel development, and allows for easier testing and maintenance.

Limitations: Can lead to performance bottlenecks if layers are not properly designed, may introduce communication overhead between layers, and may become overly complex with too many layers.